

Relaciones Uno a Muchos (1:N)

Objetivos de Aprendizaje

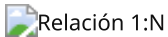
Al finalizar este tema, serás capaz de:

-  Entender qué es una relación uno a muchos
-  Implementar `ForeignKey` en SQLAlchemy
-  Usar `relationship()` para navegar entre modelos
-  Configurar `back_populates` para relaciones bidireccionales

Contenido

1. ¿Qué es una Relación 1:N?

Una relación **uno a muchos** conecta un registro de una tabla con múltiples registros de otra:



Ejemplos del mundo real:

- Un **Author** tiene muchos **Posts** (1 autor → N posts)
- Un **User** tiene muchas **Orders** (1 usuario → N pedidos)
- Un **Department** tiene muchos **Employees** (1 depto → N empleados)

2. Implementación en SQLAlchemy

Paso 1: Definir el lado "Uno" (Parent)

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Author(Base):
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))

    # Relación: Un autor tiene muchos posts
    posts: Mapped[list["Post"]] = relationship(
        back_populates="author"
    )
```

Paso 2: Definir el lado "Muchos" (Child)

```
from sqlalchemy import String, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
```

```

title: Mapped[str] = mapped_column(String(200))
content: Mapped[str]

# Foreign Key: referencia al autor
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

# Relación inversa: cada post pertenece a un autor
author: Mapped["Author"] = relationship(
    back_populates="posts"
)

```

3. Componentes Clave

ForeignKey

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

- Define la **columna** que almacena la referencia
- El string `"authors.id"` es `"nombre_tabla.columna"`
- Se coloca en el lado **"muchos"** (Post)

relationship()

```
posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

- Define cómo **navegar** entre objetos Python
- **NO crea columnas** en la base de datos
- `Mapped[list["Post"]]` → un autor tiene una **lista** de posts
- `Mapped["Author"]` → un post tiene **un** autor

back_populates

Conecta ambos lados de la relación:

```

# En Author
posts: Mapped[list["Post"]] = relationship(back_populates="author")

# En Post
author: Mapped["Author"] = relationship(back_populates="posts")

```

- Los nombres deben coincidir cruzados
- Permite navegar en **ambas direcciones**

4. Usando la Relación

Crear Post con Author

```

from sqlalchemy.orm import Session

def create_post_for_author(
    db: Session,
    author_id: int,
    title: str,
    content: str
)

```

```

) -> Post:
    post = Post(
        title=title,
        content=content,
        author_id=author_id # Solo necesitas el ID
    )
    db.add(post)
    db.commit()
    db.refresh(post)
    return post

```

Navegar de Author a Posts

```

def get_author_with_posts(db: Session, author_id: int) -> Author | None:
    author = db.get(Author, author_id)
    if author:
        # Acceder a los posts es automático
        print(f"Author: {author.name}")
        print(f"Posts: {len(author.posts)}")
        for post in author.posts:
            print(f"  - {post.title}")
    return author

```

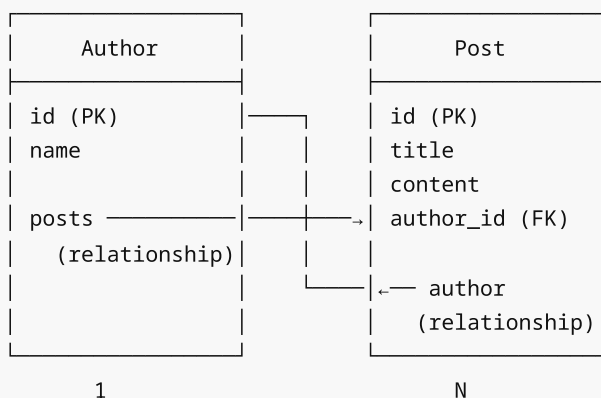
Navegar de Post a Author

```

def get_post_with_author(db: Session, post_id: int) -> Post | None:
    post = db.get(Post, post_id)
    if post:
        # Acceder al autor es automático
        print(f"Post: {post.title}")
        print(f"Author: {post.author.name}")
    return post

```

5. Diagrama de la Relación



6. Opciones Comunes de relationship()

```

# Cascade delete: eliminar posts cuando se elimina el autor
posts: Mapped[list["Post"]] = relationship(

```

```

        back_populates="author",
        cascade="all, delete-orphan"
    )

# Lazy loading (por defecto): carga posts cuando accedes
posts: Mapped[list["Post"]] = relationship(
    back_populates="author",
    lazy="select" # Valor por defecto
)

# Eager loading: carga posts junto con el autor
posts: Mapped[list["Post"]] = relationship(
    back_populates="author",
    lazy="joined" # Siempre hace JOIN
)

```

7. Ejemplo Completo

```

# models/author.py
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Author(Base):
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)

    # Relación 1:N con Post
    posts: Mapped[list["Post"]] = relationship(
        back_populates="author",
        cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"Author(id={self.id}, name='{self.name}')"

# models/post.py
from datetime import datetime
from sqlalchemy import String, Text, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

    # Foreign Key

```

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

# Relación inversa
author: Mapped["Author"] = relationship(back_populates="posts")

def __repr__(self) -> str:
    return f"Post(id={self.id}, title='{self.title}')
```

✓ Checklist

- ☐ Entiendo qué es una relación 1:N
- ☐ Sé colocar `ForeignKey` en el lado correcto (muchos)
- ☐ Puedo usar `relationship()` con `back_populates`
- ☐ Puedo navegar de parent a children y viceversa
- ☐ Conozco las opciones `cascade` y `lazy`

[Siguiente: Relaciones Muchos a Muchos →](#)

Relaciones Muchos a Muchos (N:M)

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Entender qué es una relación muchos a muchos
- ☒ Crear tablas asociativas (junction tables)
- ☒ Usar `secondary` en `relationship()`
- ☒ Manejar relaciones N:M bidireccionales

Contenido

1. ¿Qué es una Relación N:M?

Una relación **muchos a muchos** conecta múltiples registros de ambas tablas:



Ejemplos del mundo real:

- Un **Post** tiene muchos **Tags**, y un **Tag** tiene muchos **Posts**
- Un **Student** tiene muchos **Courses**, y un **Course** tiene muchos **Students**
- Un **User** tiene muchos **Roles**, y un **Role** tiene muchos **Users**

2. La Tabla Asociativa

Las bases de datos relacionales no pueden expresar N:M directamente. Necesitamos una **tabla intermedia**:



content		
N	(tabla asociativa)	M

3. Implementación en SQLAlchemy

Paso 1: Crear la Tabla Asociativa

```
from sqlalchemy import Table, Column, Integer, ForeignKey
from database import Base

# Tabla asociativa (sin clase, solo Table)
post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", Integer, ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", Integer, ForeignKey("tags.id"), primary_key=True)
)
```

Paso 2: Definir el Modelo Post

```
from sqlalchemy import String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)

    # Relación N:M con Tag
    tags: Mapped[list["Tag"]] = relationship(
        secondary=post_tags,          # Tabla asociativa
        back_populates="posts"
    )
```

Paso 3: Definir el Modelo Tag

```
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base

class Tag(Base):
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True)

    # Relación inversa N:M con Post
    posts: Mapped[list["Post"]] = relationship(
        secondary=post_tags,
```

```
        back_populates="tags"
    )
```

4. Parámetro `secondary`

```
tags: Mapped[list["Tag"]] = relationship(
    secondary=post_tags, # ← Indica qué tabla asociativa usar
    back_populates="posts"
)
```

- `secondary` recibe la **tabla asociativa** (no un string)
- SQLAlchemy maneja automáticamente los INSERT/DELETE en la tabla intermedia
- Ambos lados de la relación usan el mismo `secondary`

5. Operaciones CRUD con N:M

Crear Post con Tags

```
def create_post_with_tags(
    db: Session,
    title: str,
    content: str,
    tag_names: list[str]
) -> Post:
    # Obtener o crear tags
    tags = []
    for name in tag_names:
        tag = db.execute(
            select(Tag).where(Tag.name == name)
        ).scalar_one_or_none()

        if not tag:
            tag = Tag(name=name)
            db.add(tag)
            tags.append(tag)

    # Crear post con tags
    post = Post(title=title, content=content, tags=tags)
    db.add(post)
    db.commit()
    db.refresh(post)
    return post

# Uso
post = create_post_with_tags(
    db,
    title="Intro to FastAPI",
    content="...",
    tag_names=["python", "fastapi", "tutorial"]
)
```

Agregar Tag a Post Existente

```
def add_tag_to_post(db: Session, post_id: int, tag_name: str) -> Post:
    post = db.get(Post, post_id)
    if not post:
        raise ValueError("Post not found")

    # Buscar o crear tag
    tag = db.execute(
        select(Tag).where(Tag.name == tag_name)
    ).scalar_one_or_none()

    if not tag:
        tag = Tag(name=tag_name)

    # Agregar a la lista (SQLAlchemy maneja la tabla asociativa)
    if tag not in post.tags:
        post.tags.append(tag)
        db.commit()

    return post
```

Eliminar Tag de Post

```
def remove_tag_from_post(db: Session, post_id: int, tag_name: str) -> Post:
    post = db.get(Post, post_id)
    if not post:
        raise ValueError("Post not found")

    # Buscar tag en los tags del post
    for tag in post.tags:
        if tag.name == tag_name:
            post.tags.remove(tag)
            db.commit()
            break

    return post
```

Obtener Posts por Tag

```
def get_posts_by_tag(db: Session, tag_name: str) -> list[Post]:
    stmt = (
        select(Post)
        .join(Post.tags)
        .where(Tag.name == tag_name)
    )
    return db.execute(stmt).scalars().all()
```

6. Tabla Asociativa con Datos Extra

A veces necesitas guardar información adicional en la relación:

```
# Ejemplo: Fecha en que se agregó el tag
from datetime import datetime
from sqlalchemy import Table, Column, Integer, ForeignKey, DateTime
```



```

post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", Integer, ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", Integer, ForeignKey("tags.id"), primary_key=True),
    Column("added_at", DateTime, default=datetime.utcnow) # ← Extra
)

```

Para acceder a estos datos extra, necesitas un **Association Object** (tema más avanzado).

7. Ejemplo Completo

```

# models/associations.py
from sqlalchemy import Table, Column, Integer, ForeignKey
from database import Base

post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", Integer, ForeignKey("posts.id", ondelete="CASCADE"), primary_key=True),
    Column("tag_id", Integer, ForeignKey("tags.id", ondelete="CASCADE"), primary_key=True)
)

# models/tag.py
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base
from models.associations import post_tags

class Tag(Base):
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True, index=True)

    posts: Mapped[list["Post"]] = relationship(
        secondary=post_tags,
        back_populates="tags"
    )

    def __repr__(self) -> str:
        return f"Tag(id={self.id}, name='{self.name}')"

# models/post.py
from datetime import datetime
from sqlalchemy import String, Text, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column, relationship
from database import Base
from models.associations import post_tags

class Post(Base):
    __tablename__ = "posts"

```

```

id: Mapped[int] = mapped_column(primary_key=True)
title: Mapped[str] = mapped_column(String(200))
content: Mapped[str] = mapped_column(Text)
created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

# FK a Author (relación 1:N)
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
author: Mapped["Author"] = relationship(back_populates="posts")

# Relación N:M con Tag
tags: Mapped[list["Tag"]] = relationship(
    secondary=post_tags,
    back_populates="posts"
)

def __repr__(self) -> str:
    return f"Post(id={self.id}, title='{self.title}')"

```

✓ Checklist

- ☐ Entiendo qué es una relación N:M
- ☐ Sé crear tablas asociativas con `Table()`
- ☐ Puedo usar `secondary` en `relationship()`
- ☐ Puedo agregar/eliminar elementos de la relación
- ☐ Puedo hacer queries que involucren N:M

[← Anterior: Relaciones 1:N](#) | [Siguiente: Queries con Relaciones →](#)

Queries con Relaciones

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Realizar JOINS entre tablas relacionadas
- ☒ Entender lazy loading vs eager loading
- ☒ Optimizar queries para evitar el problema N+1
- ☒ Filtrar y ordenar por campos de relaciones

Contenido

1. El Problema N+1

Cuando accedes a relaciones sin cuidado, puedes generar muchas queries:

```

# ✗ PROBLEMA N+1
def list_posts_with_authors(db: Session) -> list[Post]:
    posts = db.execute(select(Post)).scalars().all() # 1 query

    for post in posts:
        print(f"{post.title} by {post.author.name}") # N queries!

```

```
return posts
```

Si tienes 100 posts, esto genera **101 queries** (1 + 100).

2. Lazy Loading vs Eager Loading

Lazy Loading (Por defecto)

```
# Las relaciones se cargan cuando las accedes
post = db.get(Post, 1)          # Query 1: SELECT * FROM posts WHERE id=1
print(post.author.name)        # Query 2: SELECT * FROM authors WHERE id=...
print(post.tags)               # Query 3: SELECT * FROM tags JOIN post_tags...
```

-  Eficiente si no siempre necesitas las relaciones
-  Genera N+1 si iteras sobre muchos objetos

Eager Loading

Carga las relaciones **junto con** el objeto principal:

```
from sqlalchemy.orm import joinedload, selectinload

# JOIN en una sola query
stmt = select(Post).options(joinedload(Post.author))
posts = db.execute(stmt).scalars().unique().all()
```

3. Métodos de Eager Loading

`joinedload()` - Para relaciones 1:N (lado "uno")

```
from sqlalchemy.orm import joinedload

# Cargar post con su autor (1 query con JOIN)
stmt = (
    select(Post)
    .options(joinedload(Post.author))
    .where(Post.id == post_id)
)
post = db.execute(stmt).scalar_one_or_none()

# SQL generado:
# SELECT posts.*, authors.*
# FROM posts
# LEFT JOIN authors ON posts.author_id = authors.id
# WHERE posts.id = ?
```

`selectinload()` - Para relaciones 1:N (lado "muchos") y N:M

```
from sqlalchemy.orm import selectinload

# Cargar autor con sus posts (2 queries optimizadas)
stmt = (
    select(Author)
```

```

        .options(selectinload(Author.posts))
        .where(Author.id == author_id)
    )
    author = db.execute(stmt).scalar_one_or_none()

# SQL generado:
# Query 1: SELECT * FROM authors WHERE id = ?
# Query 2: SELECT * FROM posts WHERE author_id IN (?)

```

Combinando ambos

```

# Cargar posts con autor Y tags
stmt = (
    select(Post)
    .options(
        joinedload(Post.author),      # JOIN para autor
        selectinload(Post.tags)      # SELECT IN para tags
    )
)
posts = db.execute(stmt).scalars().unique().all()

```

4. JOINS Explícitos para Filtrar

Cuando quieres **filtrar** por campos de la relación:

```

from sqlalchemy import select

# Posts de un autor específico
stmt = (
    select(Post)
    .join(Post.author) # JOIN explícito
    .where(Author.name == "John Doe")
)
posts = db.execute(stmt).scalars().all()

# SQL:
# SELECT posts.* FROM posts
# JOIN authors ON posts.author_id = authors.id
# WHERE authors.name = 'John Doe'

```

Filtrar por relación N:M

```

# Posts con tag específico
stmt = (
    select(Post)
    .join(Post.tags) # JOIN a través de tabla asociativa
    .where(Tag.name == "python")
)
posts = db.execute(stmt).scalars().all()

```

Posts con múltiples tags (AND)

```
# Posts que tienen AMBOS tags
stmt = (
    select(Post)
    .join(Post.tags)
    .where(Tag.name.in_(["python", "fastapi"]))
    .group_by(Post.id)
    .having(func.count(Tag.id) == 2) # Debe tener los 2
)
```

5. Ordenar por Relación

```
from sqlalchemy import select, desc

# Posts ordenados por nombre del autor
stmt = (
    select(Post)
    .join(Post.author)
    .order_by(Author.name, desc(Post.created_at))
)
posts = db.execute(stmt).scalars().all()
```

6. Subqueries

Para queries más complejas:

```
from sqlalchemy import select, func

# Autores con más de 5 posts
subq = (
    select(Post.author_id, func.count(Post.id).label("post_count"))
    .group_by(Post.author_id)
    .having(func.count(Post.id) > 5)
    .subquery()
)

stmt = (
    select(Author)
    .join(subq, Author.id == subq.c.author_id)
)
prolific_authors = db.execute(stmt).scalars().all()
```

7. Ejemplos Prácticos

Listar Posts con Autor y Tags (Optimizado)

```
def list_posts_optimized(db: Session, skip: int = 0, limit: int = 10) -> list[Post]:
    """Lista posts con sus relaciones cargadas eficientemente"""
    stmt = (
        select(Post)
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
    )
```

```

    )
    .order_by(desc(Post.created_at))
    .offset(skip)
    .limit(limit)
)
return db.execute(stmt).scalars().unique().all()

```

Buscar Posts por Tag

```

def get_posts_by_tag(db: Session, tag_name: str) -> list[Post]:
    """Obtiene posts que tienen un tag específico"""
    stmt = (
        select(Post)
        .join(Post.tags)
        .where(Tag.name == tag_name)
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
        .order_by(desc(Post.created_at))
    )
    return db.execute(stmt).scalars().unique().all()

```

Estadísticas de Autor

```

def get_author_stats(db: Session, author_id: int) -> dict:
    """Obtiene estadísticas de un autor"""
    author = db.get(Author, author_id)
    if not author:
        return None

    # Contar posts
    post_count = db.execute(
        select(func.count(Post.id)).where(Post.author_id == author_id)
    ).scalar()

    # Tags únicos usados
    stmt = (
        select(func.count(func.distinct(Tag.id)))
        .select_from(Post)
        .join(Post.tags)
        .where(Post.author_id == author_id)
    )
    unique_tags = db.execute(stmt).scalar()

    return {
        "author": author.name,
        "total_posts": post_count,
        "unique_tags": unique_tags
    }

```

8. Resumen de Estrategias

Situación	Estrategia	Método
Cargar 1 objeto relacionado	Eager loading	<code>joinedload()</code>
Cargar lista de objetos relacionados	Eager loading	<code>selectinload()</code>
Filtrar por campo de relación	JOIN explícito	<code>.join().where()</code>
Ordenar por campo de relación	JOIN explícito	<code>.join().order_by()</code>
Contar relacionados	Subquery o func	<code>func.count()</code>

9. `.unique()` Importante

Cuando usas `joinedload()`, puedes obtener duplicados:

```
# ❌ Puede tener duplicados
posts = db.execute(stmt).scalars().all()

# ✅ Elimina duplicados
posts = db.execute(stmt).scalars().unique().all()
```

✅ Checklist

- ☐ Entiendo el problema N+1
- ☐ Sé cuándo usar `joinedload()` vs `selectinload()`
- ☐ Puedo hacer JOINS explícitos para filtrar
- ☐ Sé usar `.unique()` cuando es necesario
- ☐ Puedo ordenar por campos de relaciones

[← Anterior: Relaciones N:M](#) | [Siguiente: Introducción Service Layer →](#)

Introducción al Service Layer

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Entender qué es el Service Layer y por qué usarlo
- ☒ Identificar problemas de código sin separación de capas
- ☒ Conocer la arquitectura de capas en FastAPI
- ☒ Diferenciar responsabilidades de cada capa

Contenido

1. ¿Qué es el Service Layer?

El **Service Layer** es un patrón arquitectónico que **separa la lógica de negocio** de los endpoints HTTP.

 Service Layer

2. El Problema: Endpoints "Fat"

Sin Service Layer, todo está en el endpoint:

```
# ❌ MAL: Endpoint "gordo" con todo mezclado
@app.post("/posts", response_model=PostResponse)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    # Validación de negocio
    if len(post.title) < 5:
        raise HTTPException(400, "Title too short")

    # Verificar que el autor existe
    author = db.get(Author, post.author_id)
    if not author:
        raise HTTPException(404, "Author not found")

    # Verificar que el autor puede publicar
    if not author.is_active:
        raise HTTPException(403, "Author is not active")

    # Procesar tags
    tags = []
    for tag_name in post.tag_names:
        tag = db.execute(
            select(Tag).where(Tag.name == tag_name)
        ).scalar_one_or_none()
        if not tag:
            tag = Tag(name=tag_name)
            db.add(tag)
        tags.append(tag)

    # Crear el post
    db_post = Post(
        title=post.title,
        content=post.content,
        author_id=post.author_id,
        tags=tags
    )
    db.add(db_post)
    db.commit()
    db.refresh(db_post)

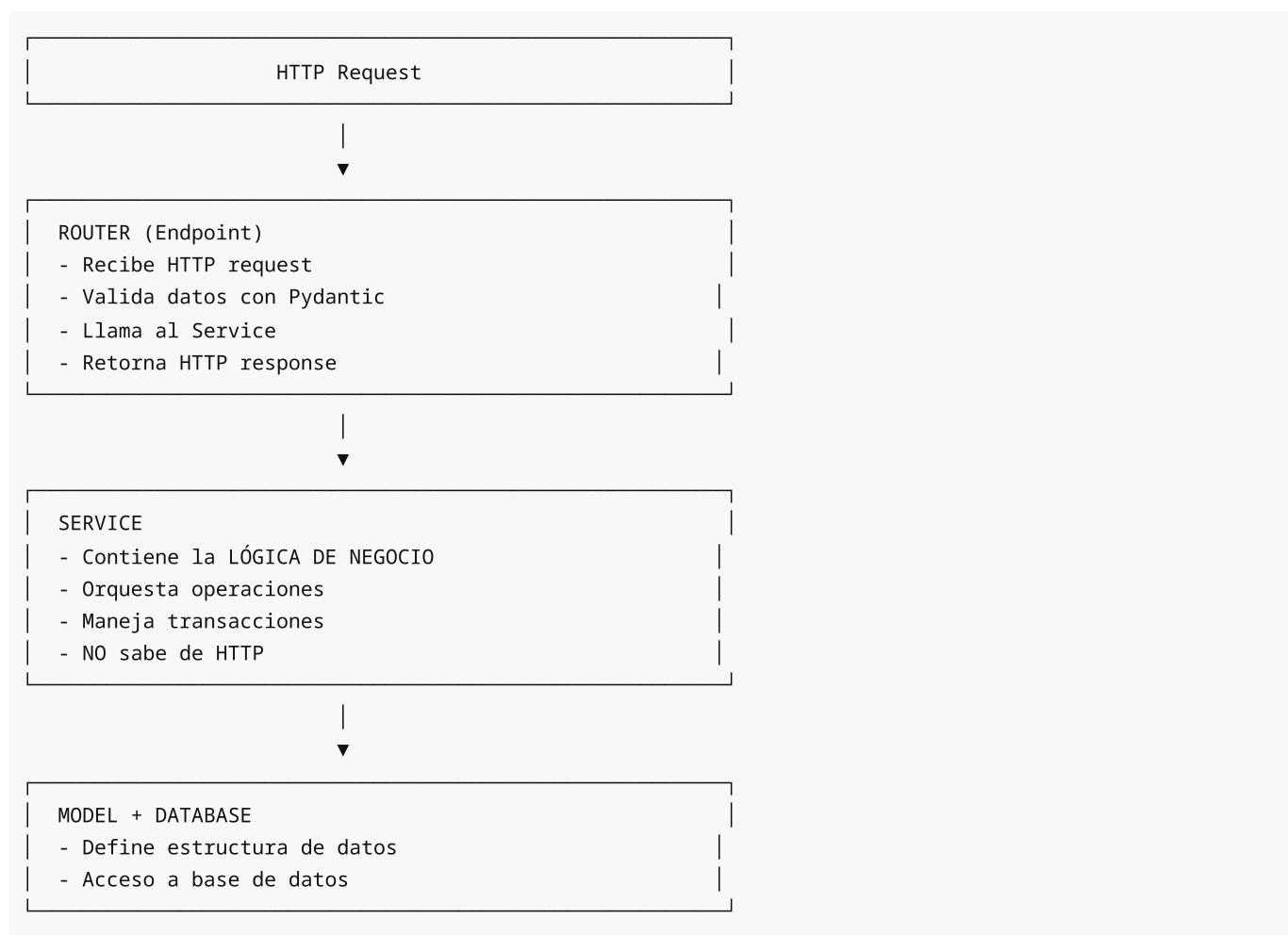
    # Enviar notificación (más lógica de negocio)
    send_notification(author.email, f"Post '{post.title}' created")

    return db_post
```

Problemas:

- 🍷 Difícil de testear (necesitas HTTP para probar lógica)
- 🍷 Difícil de reutilizar (no puedes crear posts desde otro lugar)
- 🍷 Difícil de mantener (50+ líneas de código mezclado)
- 🍷 Difícil de entender (HTTP + negocio + DB mezclados)

3. La Solución: Separación de Capas



4. Beneficios del Service Layer

Beneficio	Descripción
Testabilidad	Puedes testear lógica sin HTTP
Reutilización	La misma lógica para API, CLI, workers
Mantenibilidad	Código organizado por responsabilidad
Claridad	Fácil entender qué hace cada parte
Escalabilidad	Fácil agregar nuevas features

5. Estructura de Carpetas

```
src/
├─ main.py           # FastAPI app, configuración
├─ database.py       # Engine, Session, Base
├─
├─ routers/         # 🌐 Capa HTTP
│   ├── __init__.py
│   ├── authors.py  # Endpoints de autores
│   └─ posts.py     # Endpoints de posts
├─
├─ services/        # 📁 Capa de Negocio
│   └─ __init__.py
```

```

|   ├── author_service.py
|   └── post_service.py
|
└── models/                # 🗄️ Capa de Datos
    ├── __init__.py
    ├── author.py
    └── post.py
└── schemas/               # 📄 DTOs (validación)
    ├── __init__.py
    ├── author.py
    └── post.py

```

6. Responsabilidades de Cada Capa

Routers (Endpoints)

```

# routers/posts.py
# ✅ SOLO maneja HTTP
@router.post("/", response_model=PostResponse, status_code=201)
def create_post(
    post: PostCreate,
    db: Session = Depends(get_db)
):
    """Endpoint HTTP - delega al service"""
    service = PostService(db)
    return service.create(post)

```

Responsabilidades:

- Definir rutas HTTP
- Validar entrada con Pydantic
- Llamar al Service
- Manejar errores HTTP
- Retornar responses

NO debe:

- Contener lógica de negocio
- Hacer queries directamente
- Conocer detalles de implementación

Services

```

# services/post_service.py
# ✅ SOLO lógica de negocio
class PostService:
    def __init__(self, db: Session):
        self.db = db

    def create(self, post_data: PostCreate) -> Post:
        """Crea un post con validaciones de negocio"""
        # Validar autor existe
        author = self.db.get(Author, post_data.author_id)
        if not author:

```

```

        raise ValueError("Author not found")

# Validar autor activo
if not author.is_active:
    raise PermissionError("Author is not active")

# Procesar tags
tags = self._get_or_create_tags(post_data.tag_names)

# Crear post
post = Post(
    title=post_data.title,
    content=post_data.content,
    author_id=post_data.author_id,
    tags=tags
)
self.db.add(post)
self.db.commit()
self.db.refresh(post)

return post

```

Responsabilidades:

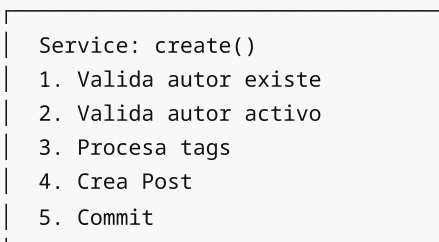
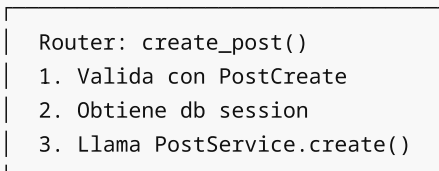
- Lógica de negocio
- Validaciones de dominio
- Orquestación de operaciones
- Manejo de transacciones

NO debe:

- Conocer detalles HTTP (códigos, headers)
- Importar FastAPI directamente
- Definir endpoints

7. Flujo de una Request

POST /posts {"title": "Hello", "author_id": 1}



Router
Retorna PostResponse (201)

8. Manejo de Errores

El Service lanza excepciones Python, el Router las convierte a HTTP:

```
# services/post_service.py
class PostService:
    def create(self, post_data: PostCreate) -> Post:
        author = self.db.get(Author, post_data.author_id)
        if not author:
            raise ValueError("Author not found") # ← Excepción Python

# routers/posts.py
@router.post("/", response_model=PostResponse)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    service = PostService(db)
    try:
        return service.create(post)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e)) # ← HTTP Error
    except PermissionError as e:
        raise HTTPException(status_code=403, detail=str(e))
```

✓ Checklist

- ☐ Entiendo por qué separar capas
- ☐ Sé qué responsabilidad tiene cada capa
- ☐ Puedo identificar endpoints "gordos"
- ☐ Entiendo el flujo Router → Service → Model

[← Anterior: Queries](#) | [Siguiendo: Implementando Servicios →](#)

Implementando Servicios

Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ☒ Crear clases de Service correctamente
- ☒ Integrar Services con FastAPI usando Dependency Injection
- ☒ Manejar errores entre capas
- ☒ Estructurar proyectos con Service Layer

Contenido

1. Anatomía de un Service

```
# services/author_service.py
from sqlalchemy import select
from sqlalchemy.orm import Session

from models.author import Author
from schemas.author import AuthorCreate, AuthorUpdate

class AuthorService:
    """
    Service para operaciones de Author.

    Contiene toda la lógica de negocio relacionada con autores.
    """

    def __init__(self, db: Session):
        """
        Inicializa el service con una sesión de DB.

        Args:
            db: Sesión de SQLAlchemy
        """
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        """Obtiene un autor por ID"""
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        """Obtiene un autor por email"""
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        """Lista autores con paginación"""
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, author_data: AuthorCreate) -> Author:
        """
        Crea un nuevo autor.

        Raises:
            ValueError: Si el email ya existe
        """
        # Validación de negocio
        if self.get_by_email(author_data.email):
            raise ValueError(f"Email {author_data.email} already registered")

        # Crear autor
        author = Author(
            name=author_data.name,
            email=author_data.email,
```

```

        bio=author_data.bio
    )
    self.db.add(author)
    self.db.commit()
    self.db.refresh(author)

    return author

def update(self, author_id: int, author_data: AuthorUpdate) -> Author:
    """
    Actualiza un autor existente.

    Raises:
        ValueError: Si el autor no existe
        ValueError: Si el nuevo email ya está en uso
    """
    author = self.get_by_id(author_id)
    if not author:
        raise ValueError(f"Author {author_id} not found")

    # Si actualiza email, verificar que no exista
    update_dict = author_data.model_dump(exclude_unset=True)
    if "email" in update_dict:
        existing = self.get_by_email(update_dict["email"])
        if existing and existing.id != author_id:
            raise ValueError(f"Email {update_dict['email']} already in use")

    # Aplicar cambios
    for key, value in update_dict.items():
        setattr(author, key, value)

    self.db.commit()
    self.db.refresh(author)

    return author

def delete(self, author_id: int) -> None:
    """
    Elimina un autor.

    Raises:
        ValueError: Si el autor no existe
    """
    author = self.get_by_id(author_id)
    if not author:
        raise ValueError(f"Author {author_id} not found")

    self.db.delete(author)
    self.db.commit()

```

2. Integración con Router

```

# routers/authors.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

```

```

from database import get_db
from schemas.author import AuthorCreate, AuthorUpdate, AuthorResponse
from services.author_service import AuthorService

router = APIRouter(prefix="/authors", tags=["authors"])

@router.post("/", response_model=AuthorResponse, status_code=status.HTTP_201_CREATED)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    """Crea un nuevo autor"""
    service = AuthorService(db)
    try:
        return service.create(author)
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=str(e)
        )

@router.get("/", response_model=list[AuthorResponse])
def list_authors(
    skip: int = 0,
    limit: int = 10,
    db: Session = Depends(get_db)
):
    """Lista autores con paginación"""
    service = AuthorService(db)
    return service.list_all(skip=skip, limit=limit)

@router.get("/{author_id}", response_model=AuthorResponse)
def get_author(author_id: int, db: Session = Depends(get_db)):
    """Obtiene un autor por ID"""
    service = AuthorService(db)
    author = service.get_by_id(author_id)

    if not author:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Author {author_id} not found"
        )

    return author

@router.put("/{author_id}", response_model=AuthorResponse)
def update_author(
    author_id: int,
    author_data: AuthorUpdate,
    db: Session = Depends(get_db)
):
    """Actualiza un autor"""
    service = AuthorService(db)
    try:

```

```

        return service.update(author_id, author_data)
    except ValueError as e:
        # Determinar código HTTP según el error
        if "not found" in str(e).lower():
            raise HTTPException(status_code=404, detail=str(e))
        raise HTTPException(status_code=409, detail=str(e))

@router.delete("/{author_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_author(author_id: int, db: Session = Depends(get_db)):
    """Elimina un autor"""
    service = AuthorService(db)
    try:
        service.delete(author_id)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e))

```

3. Service con Relaciones

```

# services/post_service.py
from sqlalchemy import select, desc
from sqlalchemy.orm import Session, joinedload, selectinload

from models.post import Post
from models.author import Author
from models.tag import Tag
from schemas.post import PostCreate, PostUpdate

class PostService:
    """Service para operaciones de Post"""

    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, post_id: int) -> Post | None:
        """Obtiene un post con sus relaciones"""
        stmt = (
            select(Post)
            .options(
                joinedload(Post.author),
                selectinload(Post.tags)
            )
            .where(Post.id == post_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(
        self,
        skip: int = 0,
        limit: int = 10,
        author_id: int | None = None,
        tag_name: str | None = None
    ) -> list[Post]:
        """Lista posts con filtros opcionales"""

```



```

stmt = (
    select(Post)
    .options(
        joinedload(Post.author),
        selectinload(Post.tags)
    )
)

# Filtrar por autor
if author_id:
    stmt = stmt.where(Post.author_id == author_id)

# Filtrar por tag
if tag_name:
    stmt = stmt.join(Post.tags).where(Tag.name == tag_name)

stmt = stmt.order_by(desc(Post.created_at)).offset(skip).limit(limit)

return self.db.execute(stmt).scalars().unique().all()

def create(self, post_data: PostCreate) -> Post:
    """
    Crea un nuevo post.

    Raises:
        ValueError: Si el autor no existe
    """
    # Validar que el autor existe
    author = self.db.get(Author, post_data.author_id)
    if not author:
        raise ValueError(f"Author {post_data.author_id} not found")

    # Obtener o crear tags
    tags = self._get_or_create_tags(post_data.tag_names or [])

    # Crear post
    post = Post(
        title=post_data.title,
        content=post_data.content,
        author_id=post_data.author_id,
        tags=tags
    )
    self.db.add(post)
    self.db.commit()
    self.db.refresh(post)

    return post

def update(self, post_id: int, post_data: PostUpdate) -> Post:
    """Actualiza un post existente"""
    post = self.db.get(Post, post_id)
    if not post:
        raise ValueError(f"Post {post_id} not found")

    update_dict = post_data.model_dump(exclude_unset=True)

```

```

# Si actualiza tags
if "tag_names" in update_dict:
    post.tags = self._get_or_create_tags(update_dict.pop("tag_names"))

# Si actualiza author_id, validar
if "author_id" in update_dict:
    author = self.db.get(Author, update_dict["author_id"])
    if not author:
        raise ValueError(f"Author {update_dict['author_id']} not found")

# Aplicar otros cambios
for key, value in update_dict.items():
    setattr(post, key, value)

self.db.commit()
self.db.refresh(post)

return post

def delete(self, post_id: int) -> None:
    """Elimina un post"""
    post = self.db.get(Post, post_id)
    if not post:
        raise ValueError(f"Post {post_id} not found")

    self.db.delete(post)
    self.db.commit()

def add_tag(self, post_id: int, tag_name: str) -> Post:
    """Agrega un tag a un post"""
    post = self.db.get(Post, post_id)
    if not post:
        raise ValueError(f"Post {post_id} not found")

    tag = self._get_or_create_tag(tag_name)

    if tag not in post.tags:
        post.tags.append(tag)
        self.db.commit()

    return post

def remove_tag(self, post_id: int, tag_name: str) -> Post:
    """Elimina un tag de un post"""
    post = self.db.get(Post, post_id)
    if not post:
        raise ValueError(f"Post {post_id} not found")

    for tag in post.tags:
        if tag.name == tag_name:
            post.tags.remove(tag)
            self.db.commit()
            break

    return post

```

```

# Métodos privados
def _get_or_create_tag(self, name: str) -> Tag:
    """Obtiene un tag o lo crea si no existe"""
    stmt = select(Tag).where(Tag.name == name)
    tag = self.db.execute(stmt).scalar_one_or_none()

    if not tag:
        tag = Tag(name=name)
        self.db.add(tag)
        self.db.flush() # Para obtener el ID sin commit

    return tag

def _get_or_create_tags(self, names: list[str]) -> list[Tag]:
    """Obtiene o crea múltiples tags"""
    return [self._get_or_create_tag(name) for name in names]

```

4. Excepciones Personalizadas (Opcional)

Para mejor organización de errores:

```

# exceptions.py
class ServiceError(Exception):
    """Base exception for service errors"""
    pass

class NotFoundError(ServiceError):
    """Resource not found"""
    pass

class DuplicateError(ServiceError):
    """Duplicate resource"""
    pass

class ValidationError(ServiceError):
    """Business validation failed"""
    pass

# services/author_service.py
from exceptions import NotFoundError, DuplicateError

class AuthorService:
    def create(self, author_data: AuthorCreate) -> Author:
        if self.get_by_email(author_data.email):
            raise DuplicateError(f"Email {author_data.email} already registered")
        # ...

# routers/authors.py
from exceptions import NotFoundError, DuplicateError

@router.post("/", response_model=AuthorResponse)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    service = AuthorService(db)

```

```
try:
    return service.create(author)
except DuplicateError as e:
    raise HTTPException(status_code=409, detail=str(e))
except NotFoundError as e:
    raise HTTPException(status_code=404, detail=str(e))
```

5. Configurando main.py

```
# main.py
from fastapi import FastAPI
from database import engine, Base
from routers import authors, posts

# Crear tablas
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Blog API",
    description="API con Service Layer",
    version="1.0.0"
)

# Registrar routers
app.include_router(authors.router)
app.include_router(posts.router)

@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

6. Testing de Services

Una gran ventaja es poder testear sin HTTP:

```
# tests/test_author_service.py
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from database import Base
from services.author_service import AuthorService
from schemas.author import AuthorCreate

# Setup de test
engine = create_engine("sqlite:///memory:")
TestSession = sessionmaker(bind=engine)

@pytest.fixture
def db():
    Base.metadata.create_all(bind=engine)
    session = TestSession()
    yield session
```

```

session.close()
Base.metadata.drop_all(bind=engine)

def test_create_author(db):
    service = AuthorService(db)

    author_data = AuthorCreate(
        name="John Doe",
        email="john@example.com"
    )

    author = service.create(author_data)

    assert author.id is not None
    assert author.name == "John Doe"
    assert author.email == "john@example.com"

def test_create_duplicate_email_raises(db):
    service = AuthorService(db)

    author_data = AuthorCreate(name="John", email="john@example.com")
    service.create(author_data)

    # Intentar crear otro con mismo email
    with pytest.raises(ValueError, match="already registered"):
        service.create(author_data)

```

✓ Checklist

- ☐ Sé crear clases Service con métodos CRUD
- ☐ Puedo integrar Services con routers
- ☐ Manejo errores entre Service y Router
- ☐ Puedo testear Services independientemente

[← Anterior: Introducción Service Layer](#) | [Volver a README →](#)

Práctica 01: Relación Uno a Muchos

Objetivo

Implementar una relación **1:N** entre `Author` y `Post` usando SQLAlchemy 2.0.

Escenario

Crearás un modelo donde:

- Un **Author** puede tener muchos **Posts**
- Cada **Post** pertenece a un solo **Author**

Author (1) ———< Post (N)

Instrucciones

Paso 1: Configurar la Base de Datos

Abre `starter/database.py` y descomenta el código para configurar SQLAlchemy:

```
# Creamos el engine con SQLite
engine = create_engine("sqlite:///./blog.db", echo=True)

# SessionLocal para crear sesiones
SessionLocal = sessionmaker(bind=engine)

# Base para los modelos
Base = declarative_base()
```

El parámetro `echo=True` muestra las queries SQL en consola (útil para aprender).

Paso 2: Crear el Modelo Author

Abre `starter/models.py` y descomenta la clase `Author` :

```
class Author(Base):
    __tablename__ = "authors"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(100), unique=True)

    # Relación 1:N - Un autor tiene muchos posts
    posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

Puntos clave:

- `Mapped[list["Post"]]` indica que es una lista de Posts
 - `relationship()` crea la relación ORM
 - `back_populates="author"` conecta con el otro lado
-

Paso 3: Crear el Modelo Post

Descomenta la clase `Post` en el mismo archivo:

```
class Post(Base):
    __tablename__ = "posts"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)

    # Foreign Key - apunta al autor
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))

    # Relación inversa - cada post tiene un autor
    author: Mapped["Author"] = relationship(back_populates="posts")
```

Puntos clave:

- `ForeignKey("authors.id")` crea la FK en la tabla posts
- `Mapped["Author"]` (singular) indica un solo objeto
- `back_populates="posts"` conecta con la lista del Author

Paso 4: Crear las Tablas

Descomenta en `starter/main.py` :

```
from database import engine, Base
from models import Author, Post

# Crear todas las tablas
Base.metadata.create_all(bind=engine)
print("✅ Tablas creadas correctamente")
```

Ejecuta:

```
cd starter
python main.py
```

Deberías ver las queries CREATE TABLE en consola.

Paso 5: Insertar Datos

Descomenta la función `create_sample_data()` :

```
def create_sample_data():
    db = SessionLocal()

    # Crear autor
    author = Author(name="John Doe", email="john@example.com")
    db.add(author)
    db.commit()
    db.refresh(author)
    print(f"✅ Autor creado: {author.name} (ID: {author.id})")

    # Crear posts para el autor
    post1 = Post(
        title="Intro to FastAPI",
        content="FastAPI es un framework moderno...",
        author_id=author.id # Usando FK directamente
    )
    post2 = Post(
        title="SQLAlchemy 2.0",
        content="SQLAlchemy 2.0 trae muchas mejoras...",
        author=author # Usando la relación (también válido)
    )

    db.add_all([post1, post2])
    db.commit()
    print(f"✅ Posts creados: {post1.title}, {post2.title}")
```

```
db.close()
```

Paso 6: Consultar Relaciones

Descomenta `query_relationships()` :

```
def query_relationships():
    db = SessionLocal()

    # Obtener autor con sus posts
    author = db.execute(
        select(Author).where(Author.email == "john@example.com")
    ).scalar_one()

    print(f"\n📝 Posts de {author.name}:")
    for post in author.posts: # Acceso via relación
        print(f"    - {post.title}")

    # Obtener post y su autor
    post = db.execute(select(Post).limit(1)).scalar()
    print(f"\n👤 Autor del post '{post.title}': {post.author.name}")

    db.close()
```

Paso 7: Ejecutar Todo

```
python main.py
```

Salida esperada:

- ✅ Tablas creadas correctamente
- ✅ Autor creado: John Doe (ID: 1)
- ✅ Posts creados: Intro to FastAPI, SQLAlchemy 2.0

📝 Posts de John Doe:

- Intro to FastAPI
- SQLAlchemy 2.0

👤 Autor del post 'Intro to FastAPI': John Doe

✅ Checklist de Verificación

- ☐ Las tablas se crean sin errores
- ☐ Puedo crear un Author y múltiples Posts
- ☐ Puedo acceder a `author.posts` (lista)
- ☐ Puedo acceder a `post.author` (objeto)
- ☐ Entiendo dónde va la ForeignKey (en el lado "muchos")

Modifica el código para:

1. Crear un segundo autor con 3 posts
2. Listar todos los autores con la cantidad de posts de cada uno
3. Implementar `cascade="all, delete-orphan"` y probar eliminar un autor

[Siguiente: Práctica 02 - Relación N:M →](#)



Práctica 02: Relación Muchos a Muchos



Objetivo

Implementar una relación **N:M** entre `Post` y `Tag` usando una tabla asociativa.



Escenario

- Un **Post** puede tener muchos **Tags**
- Un **Tag** puede estar en muchos **Posts**

Post (N) ———< post_tags >———— Tag (M)



Instrucciones

Paso 1: Crear la Tabla Asociativa

Abre `starter/models.py` y descomenta la tabla `post_tags` :

```
from sqlalchemy import Table, Column, Integer, ForeignKey

# Tabla asociativa (no es un modelo, solo una Table)
post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", Integer, ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", Integer, ForeignKey("tags.id"), primary_key=True)
)
```

Puntos clave:

- Es una `Table` , no una clase `Base`
- Tiene dos FKs que juntas forman la PK compuesta
- No tiene modelo ORM asociado (es transparente)

Paso 2: Crear el Modelo Tag

Descomenta la clase `Tag` :

```
class Tag(Base):
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True)
```

```
# Relación N:M con Post
posts: Mapped[list["Post"]] = relationship(
    secondary=post_tags,      # ← Usa la tabla asociativa
    back_populates="tags"
)
```

Paso 3: Actualizar el Modelo Post

Añade la relación con tags al modelo Post:

```
class Post(Base):
    # ... campos existentes ...

    # Relación N:M con Tag
    tags: Mapped[list["Tag"]] = relationship(
        secondary=post_tags,
        back_populates="posts"
    )
```

Paso 4: Crear Tablas

En `starter/main.py`, descomenta:

```
Base.metadata.create_all(bind=engine)
```

Paso 5: Insertar Tags

Descomenta `create_tags()`:

```
def create_tags():
    db = SessionLocal()

    tag_names = ["python", "fastapi", "sqlalchemy", "tutorial", "backend"]

    for name in tag_names:
        tag = Tag(name=name)
        db.add(tag)

    db.commit()
    print(f"✅ Tags creados: {tag_names}")
    db.close()
```

Paso 6: Asignar Tags a Posts

Descomenta `assign_tags_to_posts()`:

```
def assign_tags_to_posts():
    db = SessionLocal()

    # Obtener un post
    post = db.execute(select(Post).limit(1)).scalar()

    # Obtener tags
```

```
python_tag = db.execute(
    select(Tag).where(Tag.name == "python")
).scalar()
fastapi_tag = db.execute(
    select(Tag).where(Tag.name == "fastapi")
).scalar()

# Asignar tags al post (como si fuera una lista)
post.tags.append(python_tag)
post.tags.append(fastapi_tag)

db.commit()
print(f"✅ Tags asignados a '{post.title}': {[t.name for t in post.tags]}")
db.close()
```

Paso 7: Consultar Relaciones N:M

Descomenta `query_many_to_many()` :

```
def query_many_to_many():
    db = SessionLocal()

    # Posts de un tag específico
    python_tag = db.execute(
        select(Tag).where(Tag.name == "python")
    ).scalar()

    print(f"\n📄 Posts con tag '{python_tag.name}':")
    for post in python_tag.posts:
        print(f"    - {post.title}")

    # Tags de un post
    post = db.execute(select(Post).limit(1)).scalar()
    print(f"\n📄 Tags del post '{post.title}':")
    for tag in post.tags:
        print(f"    - {tag.name}")

    db.close()
```

Paso 8: Agregar/Eliminar Tags

Descomenta `manage_tags()` :

```
def manage_tags():
    db = SessionLocal()

    post = db.execute(select(Post).limit(1)).scalar()

    # Agregar tag
    tutorial_tag = db.execute(
        select(Tag).where(Tag.name == "tutorial")
    ).scalar()

    if tutorial_tag not in post.tags:
```

```
post.tags.append(tutorial_tag)
print(f"✅ Tag 'tutorial' agregado")

# Eliminar tag
for tag in post.tags:
    if tag.name == "fastapi":
        post.tags.remove(tag)
        print(f"❌ Tag 'fastapi' eliminado")
        break

db.commit()
print(f"🔍 Tags actuales: {[t.name for t in post.tags]}")
db.close()
```

✅ Checklist de Verificación

- ☐ La tabla `post_tags` se crea correctamente
- ☐ Puedo crear Tags independientes
- ☐ Puedo asignar Tags a Posts con `.append()`
- ☐ Puedo eliminar Tags de Posts con `.remove()`
- ☐ Puedo acceder a `post.tags` y `tag.posts`

🎯 Reto Extra

1. Crea una función `get_or_create_tag(name)` que retorne el tag si existe o lo cree si no
2. Implementa una función para encontrar posts que tengan TODOS los tags especificados
3. Cuenta cuántos posts tiene cada tag

[← Anterior: Práctica 01](#) | [Siguiente: Práctica 03 →](#)

🔍 Práctica 03: Queries Optimizadas

🎯 Objetivo

Aprender a optimizar queries con relaciones usando **eager loading** para evitar el problema N+1.

📋 Escenario

Tienes un blog con Authors, Posts y Tags. Necesitas:

- Listar posts con su autor y tags (sin N+1)
- Filtrar posts por tag
- Ordenar por campos de relaciones

🚀 Instrucciones

Paso 1: Entender el Problema N+1

Abre `starter/main.py` y ejecuta `demonstrate_n_plus_1()` :

```
def demonstrate_n_plus_1():
    """Muestra el problema N+1"""
    db = SessionLocal()

    # 1 query para obtener posts
    posts = db.execute(select(Post)).scalars().all()

    for post in posts:
        # N queries adicionales (1 por cada post)
        print(f"{post.title} by {post.author.name}")

    db.close()
```

Observa cuántas queries se generan (mira los logs SQL).

Paso 2: Solución con `joinedload()`

Descomenta `optimized_with_joinedload()` :

```
from sqlalchemy.orm import joinedload

def optimized_with_joinedload():
    """Usa JOIN para cargar autor en una sola query"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .options(joinedload(Post.author)) # ← Eager loading
    )
    posts = db.execute(stmt).scalars().unique().all()

    for post in posts:
        # Sin queries adicionales!
        print(f"{post.title} by {post.author.name}")

    db.close()
```

Nota: Usa `.unique()` cuando usas `joinedload()` para evitar duplicados.

Paso 3: Cargar Colecciones con `selectinload()`

Para relaciones 1:N (lado muchos) o N:M, usa `selectinload()` :

```
from sqlalchemy.orm import selectinload

def load_with_selectinload():
    """Usa SELECT IN para cargar tags eficientemente"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .options(
            joinedload(Post.author),      # JOIN para 1 objeto
            selectinload(Post.tags)       # SELECT IN para lista
        )
    )
```

```

    )
)
posts = db.execute(stmt).scalars().unique().all()

for post in posts:
    tag_names = [t.name for t in post.tags]
    print(f"{post.title} - Tags: {tag_names}")

db.close()

```

Paso 4: Filtrar por Relación

Para filtrar por campos de una relación, usa JOIN explícito:

```

def filter_by_relation():
    """Filtra posts por tag"""
    db = SessionLocal()

    # Posts con tag "python"
    stmt = (
        select(Post)
        .join(Post.tags)                # JOIN explícito
        .where(Tag.name == "python")    # Filtro en Tag
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
    )
    posts = db.execute(stmt).scalars().unique().all()

    print(f"\n📝 Posts con tag 'python':")
    for post in posts:
        print(f"    - {post.title}")

    db.close()

```

Paso 5: Ordenar por Relación

```

def order_by_relation():
    """Ordena posts por nombre del autor"""
    db = SessionLocal()

    stmt = (
        select(Post)
        .join(Post.author)              # JOIN necesario para ordenar
        .order_by(Author.name, Post.title) # Ordenar por autor, luego título
        .options(joinedload(Post.author))
    )
    posts = db.execute(stmt).scalars().unique().all()

    print(f"\n📝 Posts ordenados por autor:")
    for post in posts:
        print(f"    - {post.author.name}: {post.title}")

```

```
db.close()
```

Paso 6: Query Compleja - Posts por Autor con Stats

```
from sqlalchemy import func

def author_stats():
    """Estadísticas de posts por autor"""
    db = SessionLocal()

    # Autores con cantidad de posts
    stmt = (
        select(
            Author.name,
            func.count(Post.id).label("post_count")
        )
        .join(Author.posts)
        .group_by(Author.id)
        .order_by(func.count(Post.id).desc())
    )

    results = db.execute(stmt).all()

    print(f"\n📊 Posts por autor:")
    for name, count in results:
        print(f"    - {name}: {count} posts")

    db.close()
```

Paso 7: Comparar Rendimiento

Descomenta `compare_performance()` para ver la diferencia:

```
import time

def compare_performance():
    """Compara queries con y sin optimización"""
    db = SessionLocal()

    # Sin optimización
    start = time.time()
    posts = db.execute(select(Post)).scalars().all()
    for post in posts:
        _ = post.author.name
        _ = [t.name for t in post.tags]
    lazy_time = time.time() - start

    # Con optimización
    start = time.time()
    stmt = select(Post).options(
        joinedload(Post.author),
        selectinload(Post.tags)
    )
```

```
posts = db.execute(stmt).scalars().unique().all()
for post in posts:
    _ = post.author.name
    _ = [t.name for t in post.tags]
eager_time = time.time() - start

print(f"\n🕒 Lazy loading: {lazy_time:.4f}s")
print(f"🕒 Eager loading: {eager_time:.4f}s")

db.close()
```

✅ Checklist de Verificación

- ☐ Entiendo el problema N+1
- ☐ Sé cuándo usar `joinedload()` vs `selectinload()`
- ☐ Puedo filtrar por campos de relaciones
- ☐ Puedo ordenar por campos de relaciones
- ☐ Uso `.unique()` cuando es necesario

🎯 Reto Extra

- Encuentra los 3 tags más usados
- Lista posts que NO tienen tags
- Encuentra autores que no tienen posts

[← Anterior: Práctica 02](#) | [Siguiente: Práctica 04 →](#)

🏗️ Práctica 04: Refactor a Service Layer

🎯 Objetivo

Refactorizar una API de endpoints "gordos" a una arquitectura con **Service Layer**.

📋 Escenario

Tienes una API con toda la lógica en los endpoints. Tu tarea es:

- Crear un Service para Authors
- Crear un Service para Posts
- Refactorizar los endpoints para usar los Services

📁 Estructura Objetivo

```
starter/
├─ main.py           # App FastAPI + routers
├─ database.py       # Engine + Session
├─ models.py         # SQLAlchemy models
├─ schemas.py        # Pydantic schemas
├─ services/         # 📁 NUEVA CARPETA
│   └─ __init__.py
│   └─ author_service.py
```



```
| └─ post_service.py
└─ routers/          # 🌐 Endpoints refactorizados
    └─ __init__.py
    └─ authors.py
    └─ posts.py
```

Instrucciones

Paso 1: Revisar el Código Actual

Abre `starter/main_before.py` y observa el endpoint "gordo":

```
@app.post("/posts")
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    # Validación de negocio
    author = db.get(Author, post.author_id)
    if not author:
        raise HTTPException(404, "Author not found")

    # Procesar tags
    tags = []
    for tag_name in post.tag_names:
        tag = db.execute(select(Tag).where(Tag.name == tag_name)).scalar()
        if not tag:
            tag = Tag(name=tag_name)
            db.add(tag)
            tags.append(tag)

    # Crear post
    db_post = Post(title=post.title, content=post.content, author_id=post.author_id, tags=tags)
    db.add(db_post)
    db.commit()
    db.refresh(db_post)
    return db_post
```

Problemas identificados:

- 🍷 Lógica de negocio mezclada con HTTP
- 🍷 Difícil de testear sin HTTP
- 🍷 No reutilizable

Paso 2: Crear AuthorService

Abre `starter/services/author_service.py` y descomenta:

```
class AuthorService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()
```

```

def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
    stmt = select(Author).offset(skip).limit(limit)
    return self.db.execute(stmt).scalars().all()

def create(self, data: AuthorCreate) -> Author:
    if self.get_by_email(data.email):
        raise ValueError(f"Email {data.email} already exists")

    author = Author(name=data.name, email=data.email)
    self.db.add(author)
    self.db.commit()
    self.db.refresh(author)
    return author

def delete(self, author_id: int) -> None:
    author = self.get_by_id(author_id)
    if not author:
        raise ValueError(f"Author {author_id} not found")

    self.db.delete(author)
    self.db.commit()

```

Paso 3: Crear PostService

Abre `starter/services/post_service.py` y descomenta:

```

class PostService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, post_id: int) -> Post | None:
        stmt = (
            select(Post)
            .options(joinedload(Post.author), selectinload(Post.tags))
            .where(Post.id == post_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def create(self, data: PostCreate) -> Post:
        # Validar autor
        author = self.db.get(Author, data.author_id)
        if not author:
            raise ValueError(f"Author {data.author_id} not found")

        # Procesar tags
        tags = self._get_or_create_tags(data.tag_names or [])

        # Crear post
        post = Post(
            title=data.title,
            content=data.content,
            author_id=data.author_id,
            tags=tags
        )

```

```

        self.db.add(post)
        self.db.commit()
        self.db.refresh(post)
        return post

def _get_or_create_tags(self, names: list[str]) -> list[Tag]:
    tags = []
    for name in names:
        tag = self.db.execute(
            select(Tag).where(Tag.name == name)
        ).scalar_one_or_none()
        if not tag:
            tag = Tag(name=name)
            self.db.add(tag)
        tags.append(tag)
    return tags

```

Paso 4: Refactorizar Router de Authors

Abre `starter/routers/authors.py` y descomenta:

```

@router.post("/", response_model=AuthorResponse, status_code=201)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    service = AuthorService(db)
    try:
        return service.create(author)
    except ValueError as e:
        raise HTTPException(status_code=409, detail=str(e))

@router.get("/{author_id}", response_model=AuthorResponse)
def get_author(author_id: int, db: Session = Depends(get_db)):
    service = AuthorService(db)
    author = service.get_by_id(author_id)
    if not author:
        raise HTTPException(status_code=404, detail="Author not found")
    return author

```

Paso 5: Refactorizar Router de Posts

Abre `starter/routers/posts.py` y descomenta:

```

@router.post("/", response_model=PostResponse, status_code=201)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    service = PostService(db)
    try:
        return service.create(post)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e))

@router.get("/{post_id}", response_model=PostResponse)
def get_post(post_id: int, db: Session = Depends(get_db)):
    service = PostService(db)

```

```
post = service.get_by_id(post_id)
if not post:
    raise HTTPException(status_code=404, detail="Post not found")
return post
```

Paso 6: Actualizar main.py

```
from fastapi import FastAPI
from database import engine, Base
from routers import authors, posts

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Blog API con Service Layer")

app.include_router(authors.router)
app.include_router(posts.router)
```

Paso 7: Probar la API

```
cd starter
uvicorn main:app --reload
```

Prueba en `/docs` :

1. Crear un autor
2. Crear un post con tags
3. Obtener el post con sus relaciones

✅ Checklist de Verificación

- ☐ Services creados en carpeta `services/`
- ☐ Routers refactorizados para usar Services
- ☐ Lógica de negocio está en Services, no en Routers
- ☐ Services lanzan excepciones Python, Routers las convierten a HTTP
- ☐ La API funciona igual que antes del refactor

🎯 Reto Extra

1. Agrega método `update()` a `AuthService`
2. Agrega método `add_tag()` y `remove_tag()` a `PostService`
3. Crea tests unitarios para los Services (sin HTTP)

[← Anterior: Práctica 03](#) | [Volver al README →](#)



Proyecto Semana 06: API con Service Layer y Relaciones

N:M

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

 **IMPORTANTE:** Cada aprendiz trabaja sobre un dominio diferente.

Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que **NO** está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Entity A	Item	{YourEntityA}
Entity B	Tag	{YourEntityB}
N:M Relation	Items have many Tags	{your_nm_relation}

Objetivo

Construir una **API con arquitectura por capas** implementando:

- Service Layer para lógica de negocio
- Relaciones N:M con tabla intermedia
- Eager loading para optimización
- DTOs/Schemas separados por capa

Requisitos Funcionales (Adapta a tu Dominio)

Entities (Mínimo 3: 2 principales + tabla intermedia)

```
# Ejemplo genérico (Warehouse)
class Item(Base):
    __tablename__ = "items"
    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(unique=True)
    name: Mapped[str]

    # N:M relationship
    tags: Mapped[list["Tag"]] = relationship(
        secondary="item_tags", back_populates="items"
    )

class Tag(Base):
    __tablename__ = "tags"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(unique=True)
    color: Mapped[str]

    # N:M inverse
    items: Mapped[list["Item"]] = relationship(
        secondary="item_tags", back_populates="tags"
    )

# Association table
```

```

item_tags = Table(
    "item_tags", Base.metadata,
    Column("item_id", ForeignKey("items.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True),
    Column("assigned_at", DateTime, default=datetime.utcnow),
)

```

Service Layer

```

# Ejemplo genérico
class ItemService:
    def __init__(self, db: Session):
        self.db = db

    async def add_tag_to_item(self, item_id: int, tag_id: int) -> Item:
        """Agrega un tag a un item (lógica de negocio)"""
        item = await self._get_item(item_id)
        tag = await self._get_tag(tag_id)

        if tag in item.tags:
            raise TagAlreadyAssignedError(item_id, tag_id)

        item.tags.append(tag)
        await self.db.commit()
        return item

```

Endpoints

Método	Endpoint	Descripción
POST	/items/{id}/tags/{tag_id}	Agregar tag a item
DELETE	/items/{id}/tags/{tag_id}	Quitar tag de item
GET	/items/{id}/tags	Listar tags de un item
GET	/tags/{id}/items	Listar items con un tag

Estructura del Proyecto

```

starter/
├─ main.py
├─ database.py
├─ models/
│   ├─ item.py
│   ├─ tag.py
│   └─ associations.py
├─ schemas/
├─ services/
│   ├─ item_service.py
│   └─ tag_service.py
├─ routers/
└─ pyproject.toml

```

└─ Dockerfile
└─ docker-compose.yml

✓ Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
Relación N:M funciona	15
Service Layer implementado	15
Endpoints de asociación	10
Adaptación al Dominio (35%)	
Relación N:M coherente	12
Lógica de negocio en services	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Separación de capas clara	10
Eager loading correcto	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ✗ **No copies** el ejemplo genérico "Item/Tag"
- ✓ **Diseña** una relación N:M de tu dominio
- ✓ **Implementa** lógica de negocio real

📖 Recursos

- [SQLAlchemy Many-to-Many](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

[← Volver a Prácticas](#) | [Recursos →](#)

📖 Glosario - Semana 06

Términos clave de **Relaciones SQLAlchemy** y **Service Layer**.

A

Association Table

Tabla asociativa que conecta dos entidades en una relación N:M. Solo contiene las foreign keys de ambas tablas, sin datos adicionales.

```
post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True)
)
```

B

back_populates

Parámetro de `relationship()` que **conecta bidireccionalmente** dos relaciones. Ambos lados deben declararlo apuntando al otro.

```
# En Author
posts = relationship("Post", back_populates="author")

# En Post
author = relationship("Author", back_populates="posts")
```

Business Logic

Lógica de negocio: las reglas y operaciones específicas del dominio de la aplicación. Ejemplos: validar que un email sea único, calcular descuentos, verificar permisos.

C

Cascade

Comportamiento en cascada que define qué sucede con los objetos relacionados cuando se modifica o elimina el objeto padre.

```
posts = relationship("Post", cascade="all, delete-orphan")
# Si se elimina el autor, se eliminan sus posts
```

Opciones comunes:

- `save-update` : Propagar `add()` a relacionados
- `delete` : Eliminar relacionados al eliminar padre
- `delete-orphan` : Eliminar si quedan huérfanos

D

Dependency Injection (DI)

Inyección de dependencias: patrón donde las dependencias se pasan desde fuera en lugar de crearlas internamente. FastAPI lo implementa con `Depends()`.

```
def get_author(db: Session = Depends(get_db)):
    service = AuthorService(db) # db inyectada
    return service.list_all()
```

E

Eager Loading

Carga anticipada: estrategia que carga las relaciones en la misma consulta o inmediatamente después, evitando el problema N+1.

```
# Carga autor junto con el post
stmt = select(Post).options(joinedload(Post.author))
```

F

ForeignKey

Clave foránea: columna que referencia la primary key de otra tabla. Establece la relación a nivel de base de datos.

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

J

joinedload

Estrategia de eager loading que usa **JOIN** para cargar relaciones en una sola query. Óptimo para relaciones 1:1 o N:1.

```
stmt = select(Post).options(joinedload(Post.author))
# SELECT posts.*, authors.* FROM posts JOIN authors ON ...
```

L

Lazy Loading

Carga perezosa: estrategia por defecto donde las relaciones se cargan solo cuando se acceden. Puede causar el problema N+1.

```
post = db.get(Post, 1)
# Query 1: SELECT * FROM posts WHERE id = 1

print(post.author.name)
# Query 2: SELECT * FROM authors WHERE id = ? (lazy load)
```

M

Many-to-Many (N:M)

Relación muchos a muchos: una entidad puede relacionarse con múltiples instancias de otra y viceversa. Requiere tabla asociativa.

```
Post ↔ post_tags ↔ Tag
Un post tiene muchos tags
Un tag está en muchos posts
```

Mapped

Tipo genérico de SQLAlchemy 2.0 que indica una columna mapeada con su tipo Python.

```
id: Mapped[int] = mapped_column(primary_key=True)
name: Mapped[str] = mapped_column(String(100))
bio: Mapped[str | None] = mapped_column(Text, nullable=True)
```

N

N+1 Problem

Problema N+1: antipatrón donde se ejecuta 1 query inicial + N queries adicionales (una por cada relación). Se soluciona con eager loading.

```
# ❌ N+1 Problem
authors = db.query(Author).all() # Query 1
for author in authors:
    print(author.posts) # Query 2, 3, 4... (N queries)

# ✅ Solución con eager loading
authors = db.query(Author).options(selectinload(Author.posts)).all()
# Solo 2 queries total
```

O

One-to-Many (1:N)

Relación uno a muchos: una entidad se relaciona con múltiples instancias de otra. La FK va en el lado "muchos".

```
Author (1) —> Posts (N)
Un autor tiene muchos posts
Un post pertenece a un autor
```

ORM (Object-Relational Mapping)

Mapeo objeto-relacional: técnica que convierte datos entre sistemas de tipos incompatibles (objetos Python ↔ tablas SQL).

R

relationship()

Función de SQLAlchemy que define la **relación ORM** entre modelos. No crea columnas en la base de datos.

```
# Define cómo navegar entre objetos
posts: Mapped[list["Post"]] = relationship(back_populates="author")
```

Repository Pattern

Patrón repositorio: abstrae el acceso a datos detrás de una interfaz. Similar a Service Layer pero enfocado solo en persistencia.

```
class AuthorRepository:
    def find_by_id(self, id: int) -> Author | None: ...
    def save(self, author: Author) -> Author: ...
```

S

secondary

Parámetro de `relationship()` que especifica la **tabla asociativa** en relaciones N:M.

```
tags = relationship("Tag", secondary=post_tags, back_populates="posts")
```

selectinload

Estrategia de eager loading que usa **query separada con IN**. Óptimo para relaciones 1:N o N:M.

```
stmt = select(Author).options(selectinload(Author.posts))
# Query 1: SELECT * FROM authors
# Query 2: SELECT * FROM posts WHERE author_id IN (1, 2, 3...)
```

Service Layer

Capa de servicio: patrón arquitectónico que encapsula la lógica de negocio en clases dedicadas, separándola de los endpoints HTTP.

```
Router (HTTP) → Service (Lógica) → Model (Datos)
```

Session

Objeto SQLAlchemy que gestiona la **unidad de trabajo**: tracking de cambios, transacciones, y comunicación con la base de datos.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

U

unique()

Método de SQLAlchemy 2.0 necesario después de queries con `joinedload()` para **eliminar duplicados** en colecciones.

```
stmt = select(Post).options(joinedload(Post.tags))
posts = db.execute(stmt).scalars().unique().all()
```

Comparativa: Estrategias de Loading

Estrategia	Queries	Mejor para	Ejemplo
Lazy (default)	1 + N	Relaciones poco usadas	Acceso esporádico
joinedload	1 (JOIN)	N:1, 1:1	Post → Author
selectinload	2 (IN)	1:N, N:M	Author → Posts

subqueryload	2 (subquery)	1:N complejas	Alternativa a selectin
--------------	--------------	---------------	------------------------

Comparativa: Service vs Repository

Aspecto	Service Layer	Repository Pattern
Responsabilidad	Lógica de negocio	Acceso a datos
Conoce reglas	✓ Sí	✗ No
Transacciones	Coordina	No maneja
Dependencias	Puede usar repositories	Solo DB

 **Tip:** Estos términos aparecerán constantemente en desarrollo backend. Asegúrate de entenderlos bien antes de avanzar a la siguiente semana.

Semana 06: Relaciones en SQLAlchemy + Service Layer

Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ✓ Implementar relaciones uno a muchos (1:N) entre modelos
- ✓ Implementar relaciones muchos a muchos (N:M) con tablas asociativas
- ✓ Realizar queries eficientes con joins y eager/lazy loading
- ✓ Entender el patrón Service Layer y su propósito
- ✓ Separar lógica de negocio de los endpoints (routers)
- ✓ Estructurar proyectos FastAPI con capas de responsabilidad

Contenidos

1. Teoría

#	Tema	Archivo
01	Relaciones Uno a Muchos (1:N)	01-relaciones-uno-a-muchos.md
02	Relaciones Muchos a Muchos (N:M)	02-relaciones-muchos-a-muchos.md
03	Queries con Relaciones	03-queries-con-relaciones.md
04	Introducción al Service Layer	04-introduccion-service-layer.md
05	Implementando Servicios	05-implementando-servicios.md

2. Prácticas

#	Ejercicio	Duración
01	Relación 1:N - Author → Posts	40 min
02	Relación N:M - Posts ↔ Tags	40 min
03	Queries con Relaciones	35 min

3. Proyecto

Blog API con Service Layer - API completa para un blog con:

- Authors, Posts y Tags
- Relaciones 1:N y N:M
- Arquitectura de capas (Routers → Services → Models)

 [Ver proyecto](#)

Arquitectura de Esta Semana

Esta semana introducimos el **Service Layer**:

ANTES (Week-05)

Router/Endpoint

- └ Validación (Pydantic)
- └ Lógica de negocio ← Todo mezclado
- └ Acceso a DB
- └ Response

AHORA (Week-06)

Router/Endpoint

- └ Validación (Pydantic)
- └ Llama a Service

↓

Service

- └ Lógica de negocio ← Separado
- └ Acceso a DB

Estructura de archivos

```
src/
├ main.py                # FastAPI app
├ database.py            # Engine, Session
├ routers/               # ← Endpoints HTTP
│   ├── __init__.py
│   ├── authors.py
│   └── posts.py
├ services/              # ← NUEVO: Lógica de negocio
│   ├── __init__.py
│   ├── author_service.py
│   └── post_service.py
├ models/               # SQLAlchemy Models
│   ├── __init__.py
│   ├── author.py
│   └── post.py
└ schemas/              # Pydantic Schemas
```

```
|— __init__.py
|— author.py
|— post.py
```

Distribución del Tiempo

Actividad	Tiempo
Teoría (5 temas)	2 horas
Prácticas (4 ejercicios)	2.5 horas
Proyecto	1.5 horas
Total	6 horas

Requisitos Previos

- ✓ Week-05: SQLAlchemy ORM básico
- ✓ CRUD completo con FastAPI
- ✓ Pydantic schemas
- ✓ Dependency Injection (Depends)

Entregable

Proyecto: [Blog API](#)

API de blog funcionando con:

- ☐ Relaciones 1:N y N:M implementadas
- ☐ Service Layer para lógica de negocio
- ☐ Endpoints testeados con Swagger

Navegación

← Anterior	Actual	Siguiente →
Semana 05: SQLAlchemy ORM	Semana 06	Semana 07: Repository Pattern

Recursos

- [SQLAlchemy Relationships](#)
- [FastAPI Bigger Applications](#)
- [Service Layer Pattern](#)

Rúbrica de Evaluación - Semana 06

Relaciones en SQLAlchemy + Service Layer

Distribución de Puntos

Evidencia	Porcentaje	Puntos
-----------	------------	--------

 Conocimiento	30%	30 pts
 Desempeño	40%	40 pts
 Producto	30%	30 pts
Total	100%	100 pts

Conocimiento (30 pts)

Conceptos de Relaciones (15 pts)

Criterio	Excelente (15)	Bueno (12)	Suficiente (9)	Insuficiente (0-6)
Comprensión de relaciones	Explica 1:N y N:M con precisión, entiende ForeignKey y relationship()	Entiende ambos tipos de relación	Confunde algunos conceptos	No distingue tipos de relación

Conceptos de Service Layer (15 pts)

Criterio	Excelente (15)	Bueno (12)	Suficiente (9)	Insuficiente (0-6)
Comprensión de capas	Explica separación de responsabilidades, beneficios y cuándo aplicar	Entiende la separación Router/Service	Conoce la estructura básica	No comprende el patrón

Desempeño (40 pts)

Ejercicio 01: Relación 1:N (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Implementación	ForeignKey, relationship(), back_populates correctos	Relación funciona con pequeños errores	Relación básica implementada	No funciona

Ejercicio 02: Relación N:M (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Implementación	Tabla asociativa, secondary, relación bidireccional	Funciona con limitaciones	Tabla asociativa creada	No funciona

Ejercicio 03: Queries con Relaciones (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
Queries	Joins, eager loading, filtros por relación	Queries básicas funcionan	Solo queries simples	No implementa joins

Ejercicio 04: Service Layer (10 pts)

Criterio	Completo (10)	Parcial (7)	Mínimo (4)	Incompleto (0-2)
----------	---------------	-------------	------------	------------------

Refactorización	Lógica extraída correctamente, endpoints limpios	Services funcionales con algo de lógica en routers	Estructura creada pero incompleta	No separa responsabilidades
-----------------	--	--	-----------------------------------	-----------------------------

Producto - Blog API (30 pts)

Estructura del Proyecto (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Organización	routers/, services/, models/, schemas/ correctamente separados	Estructura clara con pequeños problemas	Archivos separados pero desorganizados	Todo en un archivo

Modelos y Relaciones (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Implementación	Author→Posts (1:N), Posts↔ Tags (N:M) funcionando	Ambas relaciones con pequeños errores	Solo una relación funciona	No implementa relaciones

Services Implementados (10 pts)

Criterio	Excelente (10)	Bueno (8)	Suficiente (6)	Insuficiente (0-4)
Service Layer	AuthService, PostService, TagService con lógica separada	Services principales implementados	Al menos un service funcional	Sin service layer

Checklist de Entrega

Ejercicios

- ☐ Ejercicio 01 completado y funcional
- ☐ Ejercicio 02 completado y funcional
- ☐ Ejercicio 03 completado y funcional
- ☐ Ejercicio 04 completado y funcional

Proyecto Blog API

- ☐ Estructura de carpetas correcta
- ☐ Modelo Author con Posts (1:N)
- ☐ Modelo Post con Tags (N:M)
- ☐ Tabla asociativa post_tags
- ☐ AuthService implementado
- ☐ PostService implementado
- ☐ Endpoints funcionando
- ☐ Documentación Swagger accesible

Código

- ☐ Type hints en funciones
- ☐ Sin errores de linting

- ☐ Código comentado donde necesario
-

Notas Adicionales

Criterios de Aprobación

- Mínimo **70%** del puntaje total (70 pts)
- Al menos **50%** en cada sección
- Proyecto funcional y ejecutable

Penalizaciones

- Código sin type hints: -5 pts
- Errores de linting no corregidos: -5 pts
- Entrega tardía: -10 pts por día

Bonificación

- Tests unitarios para services: +5 pts
 - Documentación extra en código: +3 pts
-